arm Education Media

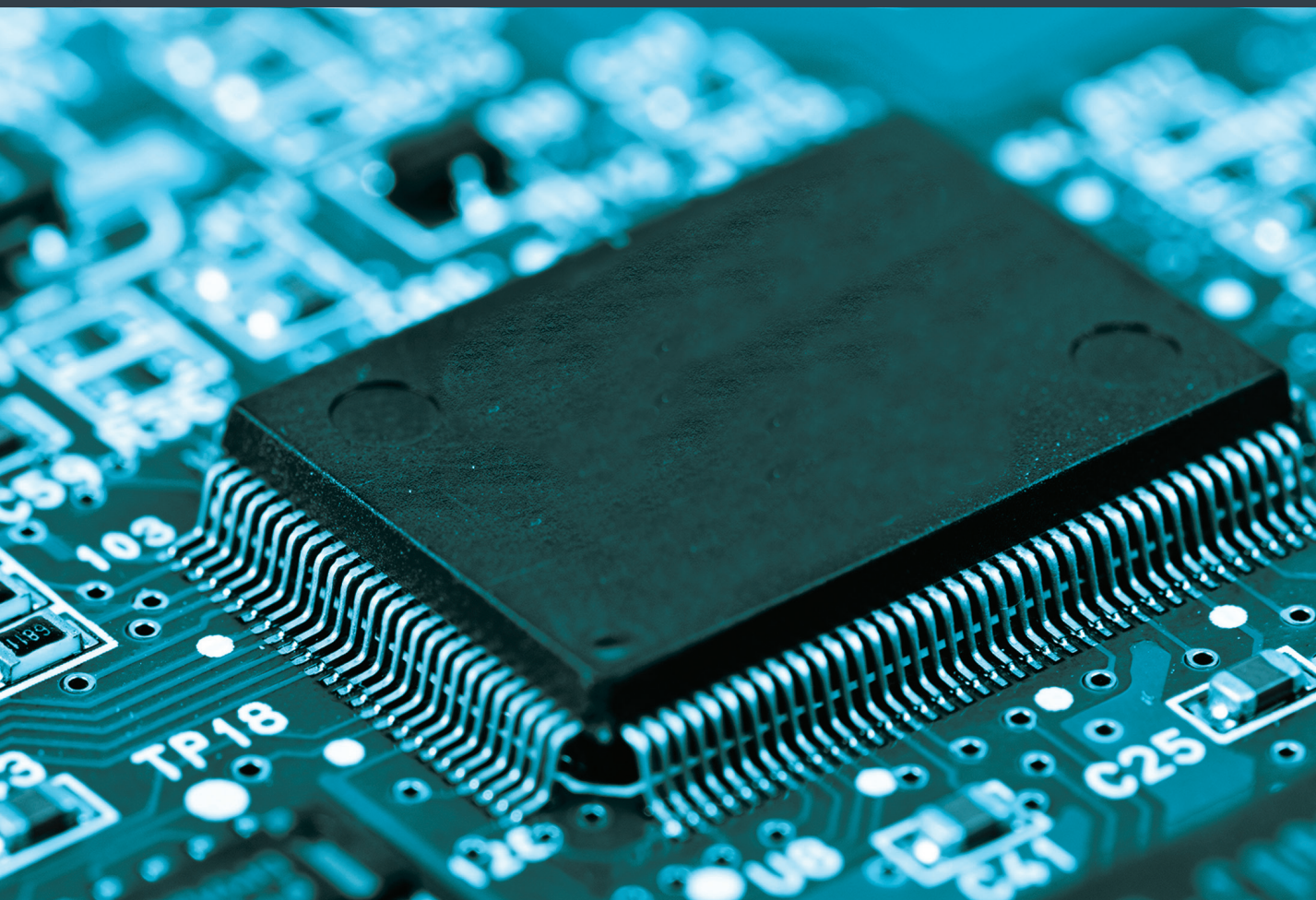
Embedded Systems Fundamentals with Arm Cortex-M based Microcontrollers

A Practical Approach

Alexander G. Dean

Nucleo-F091RC Edition

Embedded Systems Design



Embedded Systems Fundamentals with Arm® Cortex®-M based Microcontrollers

Embedded Systems Fundamentals with Arm® Cortex®-M based Microcontrollers

A Practical Approach

Alexander G. Dean

NUCLEO-F091RC EDITION

arm Education Media

Arm Education Media is an imprint of Arm Ltd., 110 Fulbourn Road, Cambridge, CBI 9NJ, UK

First published 2017

Second edition published 2021

ISBN: 978-1-911531-26-5
978-1-911531-28-9 (epub)

Version: print

British Library Cataloguing-in-Publication Data
A Catalogue record for this book is available from the British Library

Library of Congress Cataloging-in-Publication Data
A Catalog record for this book is available from the Library of Congress

For information on all Arm Education Media publications, visit our website at www.armedumedia.com

To report errors or send feedback please email edumedia@arm.com

Contents

Foreword

Alex Dean and I worked together at United Technologies in the early 1990s helping companies such as Otis, Pratt & Whitney, Carrier, and Sikorsky improve their embedded computing practices. Since then we have both chosen embedded systems as a career path, and have both done dozens of design reviews on real industry projects. Sometimes we have been able to team up on an especially important product review, which is always a blast. Whenever we chat, he tells me about his latest cool project, usually involving gadgets for his sailboat. Alex has seen the real world of embedded system design as few other professors have, and has gotten his hands dirty building real stuff. This book reflects that experience.

We are in an era in which most technologists seek to specialize, but being a good embedded system designer requires breadth across both Computer Science and Computer Engineering. Beyond that, many of the tradeoffs for embedded systems are quite different from those for desktop and enterprise systems. This book does an admirable job of covering the embedded computing design space, balancing the opposing forces of hardware versus software, depth versus breadth, and performance versus constraints. Embedded computing is usually about being highly constrained on cost, speed, memory, power, and pretty much everything else. Yet it is also about building systems that can be life-critical, or potentially put a company out of business due to the cost of a product recall if the software has a defect. Most of those who have not worked in the area do not realize how pervasive this technology is, nor how difficult it is to do well.

To give an example of how much we depend on embedded computers without necessarily realizing it, consider a server farm used for Big Data. Everyone knows there are lots of multicore big CPUs there. But there are also embedded computers and mission-critical embedded software in at least the following places in that same machine room complex: network interfaces, disk controllers, storage box controllers, board-level power supplies, rack-level power management, power-distribution switchgear, backup battery controllers, backup diesel-engine controllers, temperature monitors, air handlers, cooling compressors, cooling expansion valves, humidity controls, lighting controls, network switches, a badge-swiping system, a security video system, an alarm system, a fire-suppression system, vending machines to keep the operators happy, status monitors for many of those systems, and … well, you get the idea. Without embedded computing, the high-tech world as we know it simply would not exist. And that example is just the starting point. Embedded systems are everywhere, and I am continually astonished at the places I find not only microcontrollers, but whole teams of engineers designing and maintaining embedded systems. The difference between an embedded system that works somewhat and one that really works can easily be the difference between a product that succeeds or a product that makes the headlines (or worse) because it failed. If you doubt whether embedded system design can be a big deal if you get it wrong, consider the billions of dollars that have been spent on lawsuits because of badly written software or poor choices about what a system did.

The sweet spot for this text is for students who have seen many of the pieces before, but need a structured way to put all the pieces together. So that means they already know how to program,

how a CPU works in general, and how programs execute. But probably they have seen all this in a desktop computing environment in their introduction to computing hardware and to software systems courses. This book takes those pieces and puts them together into a coherent whole. It also helps students un-learn some of the habits that are appropriate only for big-system computing. In embedded systems, memory is not "free", nor can you simply throw more cores at a problem if you have a $1 CPU that has to run on a coin battery cell for five years. Getting these systems right requires a blend of both science and engineering, with a healthy respect for the realities of the marketplace and the messiness of interfacing to the real world.

This book gives a big picture on diverse topics, including:

- Computer organization and assembly language. If you do not understand how interrupts work, you should not be building embedded systems. (There is more, but that is a good starting point.)
- Digital design and how software interacts with bare metal hardware. Modern embedded systems are all about working with peripheral circuits to maximize system capability at minimum total cost.
- Analog I/O. The real world is not digital. (You knew that, right?)
- How to program in C. Regardless of what you think about the language, most embedded code is in C or a suspiciously C-like subset of C++, and it is the rare project that is not built on top of an existing code base.
- Tool chains and support software. Developers have to understand how compilers, real-time operating systems, and other building blocks work to use them effectively. (Hint: Most embedded micros in real products do not run Linux. Or any other OS you are likely to know.)
- Time. The computer's job is to keep up with the real world, not the other way around, and doing so creates all sorts of problems that must be understood.
- Respect and understanding for what is really going on in both the hardware and the software. The best embedded system designers understand how to exploit the strengths of both hardware and software.

The book uses the Arm Cortex-M0 processor, which has a nice selection of peripherals while still giving the feel of a resource-constrained embedded system. Beyond that, the examples have a strong dose of Alex's experience working in industry, and deal with many of the practical issues that arise in real products.

This book takes an integrated approach to putting the pieces together, rather than simply presenting the various pieces in isolation. Each chapter has well-illustrated working examples based on a real MCU evaluation board. These activities start early, with Chapter 2 showing how to read switches and light LEDs using GPIO and C code. Concurrency and responsiveness also appear early, and weave through the various examples throughout the rest of the book, working up from busy polling to preemptive task scheduling. In addition, the examples work through a progression of getting things work in a simplistic way and then improving performance by using peripherals instead of a brute force software approach. An analog waveform generator provides a running example, going from software-only timing through using DMA data transfers to the DAC.

Finally, this book emphasizes having students understand what is really going on under the hood of the compiled C code. While most embedded systems are written in C instead of assembly language these days, students need to appreciate what the compiler is doing to make their code too big, too slow, or too vulnerable to race conditions. They also need to be able to debug optimized compiled code and write the occasional low-level optimized loop that links to a C program for

that 1% of the code where speed is everything. Or even better, understand what is happening well enough to trick the compiler into generating an efficient code for them.

If, after reading this book, you are still thinking of using another book, do yourself a favor. Check the index of that other book. If the word "watchdog" does not appear, put the book down and back away from it slowly. It is not really an embedded computing book if it does not talk about watchdog timers, and you would be surprised how common that is. (Yes, Alex does cover watchdog timers in this book.) I look forward to the chance to use this book in my teaching.

For those of you who are students, pay attention to what is in this book. You have probably already looked at several of the ever-popular cut-and-paste-the-code books. They can be expedient, but you will not really learn what is going on from them, or from the books that just rehash the data sheet. Alex's book is different. It will help you put all the pieces together, so that you *understand* what you are doing, which is the great thing about having the opportunity to study and learn at a university, isn't it?

Professor Phil Koopman, Carnegie Mellon University
Pittsburgh, PA, January 2017

Preface

Introduction

It is an exciting time to develop embedded systems! Modern microcontrollers (MCUs) offer remarkable performance at a very low cost. The Internet provides an abundance of source code and documentation. The combination of inexpensive hardware platforms (e.g., Arduino, Raspberry Pi, and Beaglebone) with the right software (to abstract away details and guide users) has helped lower the barriers to embedded system development, allowing experimentation without requiring encyclopedic knowledge.

Unfortunately, these supports become shackles when trying to scale up to a larger, more complex system with tighter constraints. Industrial designers of embedded systems draw from a large toolbox of technical methods in order to meet requirements such as speed, responsiveness, cost, weight, reliability, or energy use.

Many of the hardware tools are built into the MCU: a central processing unit (CPU) to execute software, an efficient interrupt system enabling quick responses to events, fast memory to hold the program and data, and specialized hardware peripheral circuits to reduce the need for a high-speed CPU. Hardware peripherals can often signal and control each other, eliminating the need to involve software on the CPU. MCUs offer a range of low-power modes so the designer can trade off performance and power consumption as needed.

Other tools are provided by the software, which is typically written in C or C++ and compiled to run in the processor's native machine language. This avoids the run-time delays and memory overhead of interpretation or scripting. Multitasking software is scheduled on the CPU using interrupts and a scheduler, which may be cooperative (e.g., state machines) or preemptive (e.g., a real-time kernel).

To summarize, successful embedded system designs use peripherals and well-structured software on the CPU with light-weight context switching to provide responsive concurrency. This textbook aims to explain how to develop microcontroller-based embedded systems using these industry-standard methods and practice these with the most widely used processor architecture in embedded computing today: the Arm Architecture.

Challenges of Embedded Systems Education

There are several interesting challenges to learning (or teaching!) embedded systems in a college or university. First, the field builds on concepts from several areas: computer engineering (CPE), electrical engineering (EE), and computer science (CS). Some students will be able to study joint degrees, or even double or triple major, but most will not. Second, these concepts and their solutions must target embedded system design spaces, which are quite different from the mainstream general-purpose or high-performance computing design spaces covered by most courses. Third,

there are so many areas to cover that it is easy to concentrate on the familiar, which crowds out the unfamiliar. Presenting the areas with just enough detail (but not too much) can be difficult. Fourth, abstraction layers promise faster application development, but are hobbled when the developer doesn't understand the hardware beneath the abstraction. Furthermore, abstraction layers can complicate debugging and limit performance by hiding critical details among the many trivialities.

Challenge 1: Spanning Electrical and Computer Engineering and Computer Science

Successful embedded system designers need a variety of skills from CPE, EE, and CS, but not too much, and the right version given the context. These skills are typically split across ECE (electrical and computer engineering) and CS departments. To make things worse, CPE and CS courses are constantly pulled toward higher performance computers and higher levels of abstraction (to manage the increased application complexity enabled by the increased performance). This widens the gap with the embedded system design space.

The following areas in CPE and EE are the most critical for students in the field of embedded systems:

- Computer organization and assembly language programming are fundamental to an understanding of the CPU, memory, peripherals, and interrupt system.
- Digital design is necessary to understand not only how the CPU works, but more importantly to understand how peripheral circuits work. These digital circuits (e.g., GPIO, timers, DMA) provide cheap concurrency because they operate independently of the CPU. A good design will offload computationally expensive software tasks to allow a relatively slow MCU to provide precise timing and predictable performance at a low cost and with little power consumption.
- Basic analog circuit design and analysis skills are needed for adding external circuits such as LEDs, switches, and sensors. Knowing how to use an oscilloscope or logic analyzer to examine and understand the timing of events within a system is essential for effective debugging.

The following areas in CS are the most critical for students in the field:

- C language programming is necessary because it is the dominant language for programming embedded systems.
- Compilers and assembly language programming provide an understanding of how a CPU really does the work specified by the source code. Knowing how the program is compiled and structured helps with avoiding errors (e.g., preemption), debugging, designing efficient systems, and improving performance.
- Operating systems' task scheduler concepts enable students to understand how to share a single CPU among the multiple concurrent activities of the embedded system. These topics include multitasking, preemption, and prioritization. Students need to understand how to design multitasking systems using intertask communication and synchronization to avoid common bugs such as data race hazards.

Challenge 2: Targeting the ES Design Space

For each area mentioned, the practical solutions depend on the design space. The design spaces for most embedded systems are quite different from those of general purpose and high-performance computing, because of different drivers and constraints. For example:

- Computer Organization: Embedded processors typically do not need the raw speed sought in general-purpose or high-performance computing systems. As a result, they do not require high clock speeds and the deep processor pipelines and multilayer memory systems to support them.
- Operating Systems: OS courses generally target a resource-rich Linux system that features a preemptive scheduler, ample compute and memory resources, a virtual memory system with hardware support, and user and supervisor modes. This type of system does not offer the precise timing control needed for many embedded systems and is often too complex, power-hungry, and expensive. Students must be able to apply the concepts of task scheduling, synchronization, and communication to a system built on interrupts, peripherals, and a simple scheduler (whether a preemptive real-time kernel or a cooperative scheduler).
- Programming: Embedded systems use compiled languages instead of scripted or interpreted languages for reasons of predictability, efficiency, and compactness. Because of this history there is a large installed base of C/C++ development infrastructure. However, many programming curricula target Java (or even a scripted language). This abstracts away low-level and implementation issues that can make or break an embedded system.

Challenge 3: Providing Sufficient (but Not Excessive) Coverage

With all these areas to cover, it is easy to emphasize the familiar, crowding out the unfamiliar. Furthermore, the hands-on nature of embedded systems courses often slows down the progress as the student or instructor tries to get a code example working to demonstrate an important concept.

This book tries to present the areas with just enough detail (but not too much) and with practical solutions for the design space. This book does not try to present an exhaustive, complete education of all possible ways to do something. Instead, it presents the most practical options given the constraints.

Challenge 4: Providing Just Enough Abstraction

Microcontroller vendors have developed various “abstraction layers” to simplify development. These present a standard, uniform interface to hardware peripherals and system services. As systems grow more complex, these abstraction layers become more valuable. However, this is an introductory textbook which targets simple systems for clarity.

For successful embedded system development, students must master three types of information:

- How the MCU's hardware works. Explanations in the hardware manual tend to be brief for size reasons, though application notes may fill in the blanks.
- The hardware interface (control and status registers, interrupts) presented to software. The hardware manual documents this interface exhaustively.
- How to write software to use that hardware interface. This involves identifying peripheral registers and accessing specific bit fields within them.

To simplify this third item, Arm has defined the CMSIS-Core Device Peripheral Access Layer (PAL) as a clean, direct way of accessing the hardware interface [1]. Among other things, it specifies how to define data structures and address mappings for peripheral control registers and their fields, as well as macros for accessing fields within registers. STMicroelectronics has created a PAL device header file for each of its Cortex-M MCUs (e.g. `stm32f091xc.h`). The example code in Appendix A of the ST MCU reference manual uses this interface to access the peripherals [2].

To simplify development of complex embedded systems, STMicroelectronics has created an embedded software platform called STM32Cube [3]. This platform includes a wide range of software components, not just peripheral interfaces but also middleware providing an RTOS, networking, graphics, USB and so forth. STM32Cube is built on two additional abstraction layers: the STM32Cube Hardware Abstraction Layer (HAL) and low-layer APIs (LL) [4]. These layers simplify the composability and modularity of the components.

Using LL or HAL is outside the scope of this textbook. That would require the students to handle another intermediate layer of abstraction (raising complexity) when the goal is to understand how the peripherals work and then use them. Using STM32Cube with LL and HAL would be a good follow-on activity after mastering this text's material.

Notes to the Instructor

Why Use This Book?

In this textbook, I have sought to present the most important topics for embedded systems using a coherent, compelling, hands-on format.

First, the textbook uses a hands-on approach to get students excited and motivated. Each chapter has illustrated, working examples based on a real MCU evaluation board. These activities start early, with Chapter 2 showing how to read switches and light LEDs using GPIO and C code. All source code is available on GitHub [5].

Second, the textbook introduces concepts of concurrency and responsiveness early. Chapter 3 uses a running example of scanning LEDs according to switch positions to introduce concepts important for creating modular, responsive, and efficient systems. By stepping through and evaluating these improvements, the student is given a solid foundation on which to investigate real-time kernels (in a later course). Concurrency and responsiveness are introduced using the following sequence:

1. Starting with a simple program with software to poll switches, flash LEDs, and delay using busy-waiting
2. Restructuring the software into tasks
3. Scheduling the tasks cooperatively
4. Improving the responsiveness of cooperatively scheduled tasks by using state machines to break up long operations
5. Using interrupts and event-driven software to replace polling of switches
6. Replacing busy-waiting delay loops by using a timer peripheral
7. Prioritizing tasks
8. Scheduling tasks preemptively

Third, the textbook covers how to improve performance by using peripheral hardware in place of software. An analog waveform generator is used as a running example. It is introduced as an application of the digital-to-analog converter, with timing fully dependent on software execution speed. It reappears in the timer chapter, with a timer-driven periodic ISR updating the DAC to improve timing stability. The final appearance is in the DMA chapter, in which the DMA controller under timer control automatically copies data from a memory buffer to the DAC.

Fourth, the textbook covers C code as implemented in assembly language by the compiler. The main goals are to help students understand why their code is slow or large, how to make it faster or smaller, to understand preemption risks for shared data, and to help debug programs by working at both the source and object code levels. This textbook does not expect students to program in assembly language, although they may do so in a later course, given this foundation.

Course Material Linkage

This textbook is designed to be used for a one- or two-semester course introducing students to embedded systems. It complements the Efficient Embedded Systems Design and Programming Education Kit from the Arm University Program. If you are an instructor, you can receive a donation of this Education Kit by emailing university@arm.com. The Education Kit includes lecture materials and licenses to Arm's Keil MDK-ARM professional software. Students need prerequisite knowledge in C programming, digital design, and basic circuit theory.

Target Platform

This textbook targets the Arm Cortex-M0 processor, which executes the instructions of the program. The processor is a component within the microcontroller, which adds circuits to clock the processor, memory to hold the program and data, and peripheral devices that simplify programs and improve their performance. This processor is available in microcontrollers from a wide range of manufacturers.

The target platform is the Nucleo-F091RC development board from STMicroelectronics, with a list price of under $10. It uses the STM32F091RC microcontroller from the STM32F0 mainstream family. This device features a Cortex-M0 processor capable of running at up to 48 MHz, and contains 256 kB of flash ROM, 32 kB of RAM, and a wide range of peripherals. The development

board adds a USB debug interface (ST-LINK/v2-1), power supplies, and input and output devices. The board includes a temperature sensor. External devices, sensors and expansion boards can be connected to the Nucleo-F091RC to enhance its capacities. The X-NUCLEO-IKS01A1 motion MEMS and environmental sensor expansion board can be plugged into the board and provides a three-axis accelerometer and a three-axis gyroscope to sense the inclination (tilt) of the board. The expansion board can also connect to an Arduino Uno R3. External LEDs and switches can be also connected to the board.

The material in this textbook can be used with other Cortex-M0 platforms. A first step is to change `#include <stm32f091xc.h>` in the source code to specify the correct MCU. Four of the first five chapters are essentially independent of the MCU's peripherals and apply to all Cortex-M0 processors. The remaining chapters and the Appendix are closely integrated with the peripherals out of necessity. ST's other MCUs use many of the same peripherals as the STM32F0, making it easier to use those MCUs and their associated Nucleo evaluation boards.

Software Development Environment

Software examples in this textbook are written in C and compiled to run without an operating system. Arm's Keil MDK-ARM integrated development environment is used throughout the textbook. The free version of MDK-ARM supports all of the code examples in this textbook and associated course materials. Note for instructors: If the object code size limitation of the free version (currently 32 kB) is a constraint, please request a license donation from Arm for the full professional version of MDK-ARM.

Organization

The textbook is organized as follows:

Chapter 1 introduces students to the concepts of MCU-based embedded systems, and how they differ from general-purpose computers. It then introduces the Arm Cortex-M0 CPU, the STM32F091RC MCU, and the Nucleo-F091RC MCU development board.

Chapter 2 presents the general purpose I/O peripheral to provide an early, hands-on experience with reading switches and lighting LEDs using C code. It also introduces the CMSIS hardware abstraction layer, which simplifies software access to peripherals.

Chapter 3 introduces multitasking on the CPU, with the goals of improving responsiveness and software modularity while reducing CPU overhead. The interplay of interrupts, peripherals, and schedulers (both cooperative and preemptive) is examined.

Chapter 4 presents the Arm Cortex-M0 processor core, including organization, registers, memory, and instruction set. It then discusses interrupts and exceptions, including CPU response and hardware configuration. Designing software for a system with interrupts is discussed, including program design (and partitioning work), interrupt configuration, writing handlers in C, and sharing data safely given preemption.

Chapter 5 first gives an overview of the software toolchain, which translates a program from C source code to executable object code. It then shows side by side the source code and the object code the toolchain has generated to implement it. Topics covered include functions, arguments,

return values, activation records, exception handlers, control flow constructs for loops and selection, memory allocation and use, and accessing data in memory.

Chapter 6 presents analog interfacing, starting with theory and ending with practical implementations. Quantization and sampling are presented as a foundation for both digital-to-analog conversion and analog-to-digital conversion. The DAC, ADC, and analog comparator peripherals are presented and used.

Chapter 7 presents timer peripherals and their use for generating a periodic interrupt or a pulse-width modulated signal, or for measuring elapsed time or a signal's frequency. Watchdog timers, used to detect and reset an out-of-control program, are also discussed.

Chapter 8 discusses serial communication, starting with the fundamentals of data serialization, framing, error detection, media access control, and addressing. Software queues are introduced to show how to buffer data between communication ISRs and other parts of the program. Three protocols and their supporting peripherals are investigated next: SPI, asynchronous serial (UART), and I^2C. UART communication is demonstrated using the Nucleo-F091RC's debug MCU as a serial port bridge over USB to the PC. I^2C communication is demonstrated using an inertial sensor with I^2C interface.

Chapter 9 introduces the direct memory access peripheral and its ability to transfer data autonomously, offloading work from the CPU and offering dramatically improved performance. Examples include using DMA for bulk data copying, and for DAC-based analog waveform generation with precise timing.

References

[1] Arm Ltd., "CMSIS-Core (Cortex-M) Device Header File <device.h>," [Online]. Available: http://www.keil.com/pack/doc/CMSIS/Core/html/device_h_pg.html#device_access.

[2] STMicroelectronics NV, *Reference Manual RM0091: STM32F0x1/STM32F0x2/STM32F0x8*, 2017.

[3] STMicroelectronics NV, "STM32Cube Ecosystem," [Online]. Available: https://www.st.com/content/st_com/en/stm32cube-ecosystem.html.

[4] STMicroelectronics NV, *User Manual UM1785: Description of STM32F0 HAL and Low-Layer Drivers*, 2020. [Online]. Available: https://www.st.com/resource/en/user_manual/dm00122015-description-of-stm32f0-hal-and-lowlayer-drivers-stmicroelectronics.pdf.

[5] A. G. Dean, "ESF Repository," [Online]. Available: https://github.com/alexander-g-dean/ESF.git.

Acknowledgments

I would like to thank the following people for their help:

- Khaled Benkrid, Melissa Good, Liz Warman, Shuojin Hang, and the reviewers at Arm for supporting this project.
- Bill Trosky, Phil Koopman, Alan Finn, Chris McClurg, and Rocky Albano for opening so many doors to me in the embedded systems world.
- The development teams who welcomed me in as an outsider to review the embedded software for their products. They helped me understand their design goals, technical and non-technical constraints, and day-to-day challenges.
- The students who helped me sharpen my message and identify the fundamental issues at stake. Their enthusiasm and creativity are always a powerful, positive force.
- My wife and daughters for support, encouragement, and the quiet time needed to complete this work.

Author Biography

Dr. Alexander G. Dean has been a faculty member of the Department of Electrical and Computer Engineering at North Carolina State University (NCSU) since 2000. He received his BS (1991) from the University of Wisconsin, Madison, and his MS (1994) and PhD (2000) from Carnegie Mellon University.

Dr. Dean has developed four courses on embedded systems at NCSU, ranging from fundamentals to architecture and design to optimization. He has created course packages targeting five different MCU families for the university programs of three companies, including the Education Kit on Efficient Embedded Systems Design and Programming for Arm Education.

Dr. Dean's research involves using compiler, operating system, and real-time system techniques to extract more performance from commodity microcontrollers in embedded systems while reducing clock speed, energy, and memory requirements. His research also includes applying these methods for low-cost control of switch-mode power converters.

Dr. Dean has worked at United Technologies Research Center developing embedded systems and their communication network architectures. He holds three patents in the area. He has performed over 60 in-depth, on-site embedded software reviews for industry both domestically and internationally since 2001.

1